

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



PROYECTO

Segunda Entrega de Proyecto

AUTOR

Sajvin Gómez, Mario Daniel (1612921)

Backend

Salas de cine CRUD

```
const pool = require("../config/db");
const queries = require("../database/salasQueries");

const crearSala = async (req, res) => {
  let { name, rows, columns, peliculas_id } = req.body;

  // validaciones
  if (!name || !rows || !columns || !peliculas_id) {
    return res
      .status(400)
      .json({ mensaje: "Todos los campos son obligatorios" });
  }
  if (!name || typeof name !== "string" || name.trim() === "") {
    return res.status(400).json({
      message:
        "El nombre de la sala es obligatorio y debe ser un nombre válido",
    });
  }
  if (!rows || isNaN(rows) || rows <= 0) {
    return res
      .status(400)
      .json({ message: "El número de filas debe ser mayor que 0." });
  }
  if (!columns || isNaN(columns) || columns <= 0) {
    return res
      .status(400)
      .json({ message: "El número de columnas debe ser mayor que 0." });
  }
  if (!peliculas_id || isNaN(peliculas_id)) {
    return res.status(400).json({
      message: "El ID de la película es obligatorio y debe ser un número.",
    });
  }

  try {
    // verificar que la película existe
    const [pelicula] = await pool.query(queries.verificarPelículaExiste, [
      peliculas_id,
    ]);
    if (pelicula.length === 0) {
      return res
    }
  }
}
```

```

        .status(404)
        .json({ message: "La película especificada no existe" });
    }

    // crear la sala
    const [result] = await pool.query(queries.crearSala, [
        name,
        rows,
        columns,
        peliculas_id,
    ]);

    // obtener la sala recién creada, incluyendo el nombre de la película
    const [sala] = await pool.query(queries.obtenerSalaConPelicula, [
        result.insertId,
    ]);

    res
        .status(201)
        .json({ message: "Sala creada exitosadamente", sala: sala[0] });
    } catch (error) {
        console.log("Error al crear la sala: ", error);
        res.status(500).json({ message: "Error al crear la sala", error });
    }
};

const listarSalas = async (req, res) => {
    try {
        const [salas] = await pool.query(queries.listarSalas);
        res.json({ salas });
    } catch (error) {
        console.log("Error al listar las salas: ", error);
        res.status(500).json({ message: "Error al listar las salas", error });
    }
};

const actualizarSala = async (req, res) => {
    const { id } = req.params; // id de la sala
    const { name, rows, columns, peliculas_id } = req.body;

    // validaciones
    // Verificar si hay reservaciones activas para la sala
    const [reservas] = await pool.query(queries.verificarReservasEnSala,
[id]);
    if (reservas[0].total > 0) {

```

```

    return res.status(400).json({
      message:
        "No se puede actualizar la sala porque ya tiene reservaciones
activas.",
    });
  }

  if (!name || !rows || !columns || !peliculas_id) {
    return res
      .status(400)
      .json({ mensaje: "Todos los campos son obligatorios" });
  }
  if (!name || typeof name !== "string" || name.trim() === "") {
    return res.status(400).json({
      message:
        "El nombre de la sala es obligatorio y debe ser un nombre válido",
    });
  }
  if (!rows || isNaN(rows) || rows <= 0) {
    return res
      .status(400)
      .json({ message: "El número de filas debe ser mayor que 0." });
  }
  if (!columns || isNaN(columns) || columns <= 0) {
    return res
      .status(400)
      .json({ message: "El número de columnas debe ser mayor que 0." });
  }
  if (!peliculas_id || isNaN(peliculas_id)) {
    return res.status(400).json({
      message: "El ID de la película es obligatorio y debe ser un número.",
    });
  }

  try {
    // verificar que la sala existe
    const [sala] = await pool.query(queries.verificarSalaExiste, [id]);
    if (sala.length === 0) {
      return res
        .status(404)
        .json({ message: "La sala especificada no existe" });
    }

    // verificar que la película existe, antes de actualizar
    const [pelicula] = await pool.query(queries.verificarPeliculaExiste, [

```

```

        películas_id,
    ]);
    if (película.length === 0) {
        return res
            .status(404)
            .json({ message: "La película especificada no existe" });
    }

    // actualizar la sala
    await pool.query(queries.actualizarSala, [
        name,
        rows,
        columns,
        películas_id,
        id,
    ]);

    // obtener la sala actualizada, incluyendo el nombre de la película
    const [salaActualizada] = await
pool.query(queries.obtenerSalaConPelicula, [
        id,
    ]);

    res.json({
        message: "Sala actualizada existosamente",
        sala: salaActualizada[0],
    });
} catch (error) {
    console.log("Error al actualizar la sala: ", error);
    res.status(500).json({ message: "Error al actualizar la sala", error });
}
};

const eliminarSala = async (req, res) => {
    const { id } = req.params; // id de la sala

    // validar que el id se un número válido
    if (!id || isNaN(id)) {
        return res.status(400).json({
            message: "El ID de la sala es obligatorio y debe ser un número
válido.",
        });
    }

    try {

```

```

    // verificar que la sala existe, antes de eliminarla
    const [sala] = await pool.query(queries.verificarSalaExiste, [id]);
    if (sala.length === 0) {
        return res
            .status(404)
            .json({ message: "La sala especificada no existe" });
    }

    // eliminar la sala
    await pool.query(queries.eliminarSala, [id]);

    res.json({ message: "Sala eliminada exitosadamente" });
} catch (error) {
    console.log("Error al eliminar la sala: ", error);
    res.status(500).json({ message: "Error al eliminar la sala", error });
}
};

module.exports = { crearSala, listarSalas, actualizarSala, eliminarSala };

```

Endpoints para salas de Cine

```

const express = require("express");
const salasRouter = express.Router();

const { verificarToken, verificarAdmin } =
    require("../middlewares/auth");

const {
    crearSala,
    actualizarSala,
    listarSalas,
    eliminarSala,
} = require("../controllers/salas.controller");

// Definir la ruta
salasRouter.post("/crearSala", verificarToken, verificarAdmin, crearSala);
// c
salasRouter.get("/listarSalas", verificarToken, verificarAdmin,
listarSalas); // r
salasRouter.put("/actualizarSala/:id", verificarToken, verificarAdmin,
actualizarSala); // u
salasRouter.delete("/eliminarSala/:id", verificarToken, verificarAdmin,
eliminarSala); // d

```

```
module.exports = salasRouter;
```

Cómo se llama al Endpoint

```
const salasRouter = require("../salas/salas.routes");  
empaquetado.use("/salas", salasRouter);
```

Endpoints para Reservasiones

```
const pool = require("../config/db");  
const queries = require("../database/reservacionesQueries");  
  
const crearReservacion = async (req, res) => {  
  const { fecha, asientos_id } = req.body;  
  const usuarioId = req.usuario.id; // obtenido desde el token JWT  
  
  // Validar que no falte nada  
  if (!fecha || !asientos_id) {  
    return res  
      .status(400)  
      .json({ message: "Todos los campos son obligatorios." });  
  }  
  
  // Validar fecha dentro de los próximos 8 días  
  const hoy = new Date();  
  const fechaLimite = new Date(hoy);  
  fechaLimite.setDate(hoy.getDate() + 8);  
  const fechaReservacion = new Date(fecha);  
  
  if (isNaN(fechaReservacion.getTime())) {  
    return res.status(400).json({ message: "La fecha no es válida." });  
  }  
  
  if (fechaReservacion < hoy || fechaReservacion > fechaLimite) {  
    return res.status(400).json({  
      message:  
        "La reservación solo puede hacerse dentro de los próximos 8 días.",  
    });  
  }  
  
  try {  
    // Verificar si el asiento ya está reservado en esa fecha
```

```

const [resultado] = await pool.query(
  queries.verificarDisponibilidadAsiento,
  [asientos_id, fecha]
);

if (resultado.length > 0) {
  return res.status(400).json({
    message: "Este asiento ya está reservado para la fecha
seleccionada.",
  });
}

// Crear la reservación
await pool.query(queries.crearReservacion, [fecha, usuarioId,
asientos_id]);

res.status(201).json({ message: "Reservación realizada con éxito." });
} catch (error) {
  console.error("Error al crear la reservación:", error);
  res.status(500).json({ message: "Error al crear la reservación", error
});
}
};

const listarReservaciones = async (req, res) => {
  try {
    const [reservaciones] = await pool.query(queries.listarReservaciones);
    res.json(reservaciones);
  } catch (error) {
    console.error("Error al listar reservaciones:", error);
    res.status(500).json({ message: "Error al obtener reservaciones", error
});
  }
};

module.exports = { crearReservacion, listarReservaciones };

```

Endpoints para las reservaciones

```

const express = require("express");
const reservacionesRouter = express.Router();

const { verificarToken, verificarAdmin } =
require("../middlewares/auth");
const {

```



```

    crearReservacion,
    listarReservaciones,
} = require("../controllers/reservaciones.controller");

// Definir la ruta
// solo usuarios autenticados pueden crear reservaciones
reservacionesRouter.post("/crearReservacion", verificarToken,
crearReservacion);

// solo administradores pueden listar reservaciones
reservacionesRouter.get("/listarReservaciones", verificarToken,
verificarAdmin, listarReservaciones);

module.exports = reservacionesRouter;

```

Cómo se llama a los Endpoints de reservaciones

```

const reservacionesRouter = require("./reservaciones/reservaciones.routes")
empaquetado.use("/reservaciones", reservacionesRouter)

```

Middleware “Auth”

```

const jwt = require("jsonwebtoken");

const verificarAdmin = (req, res, next) => {
  console.log("Usuario autenticado: ", req.user);
  if (req.user.type !== "admin") {
    return res
      .status(403)
      .json({
        mensaje: "Acceso denegado. Se requiere el rol de administrador",
      });
  }
  next();
};

const verificarToken = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1]; // obtener el
  token desde el header

  if (!token) {

```

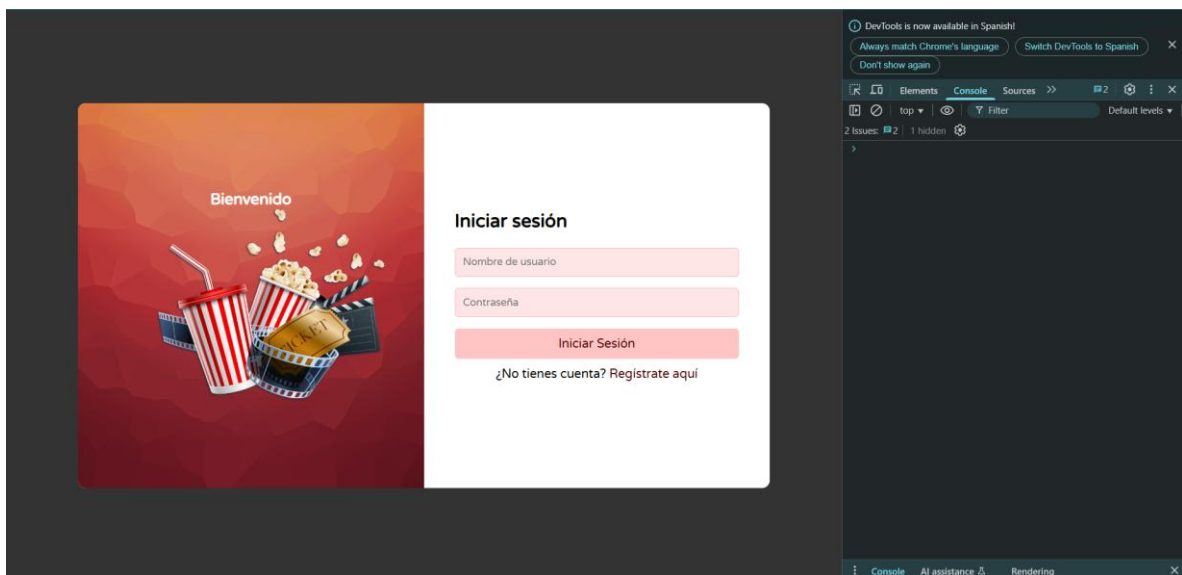
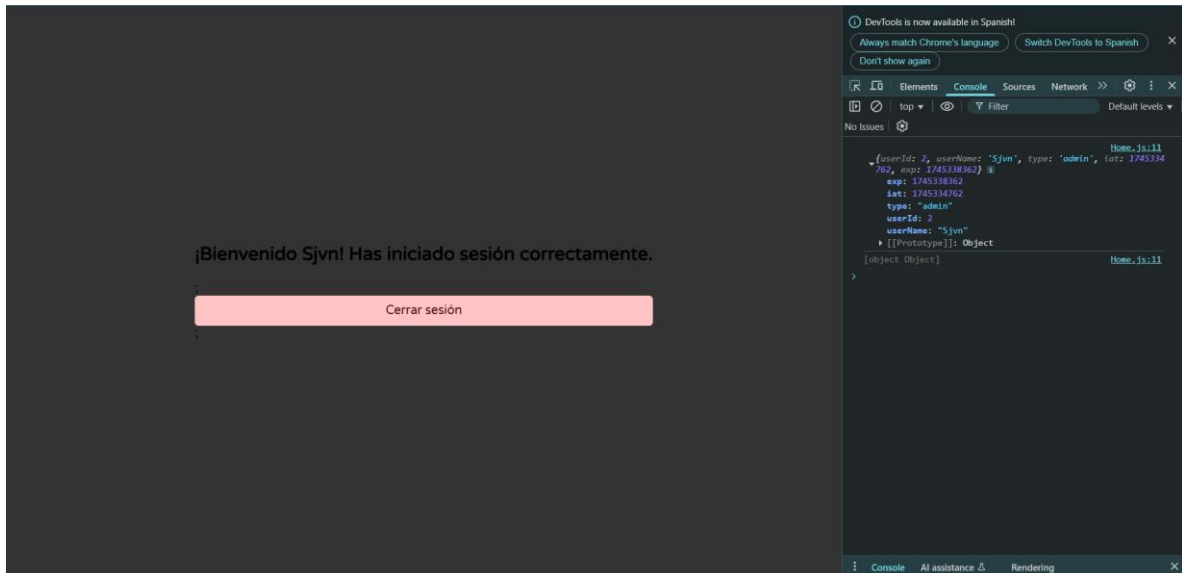
```
    return res.status(401).json({ mensaje: "No hay token" });
  }

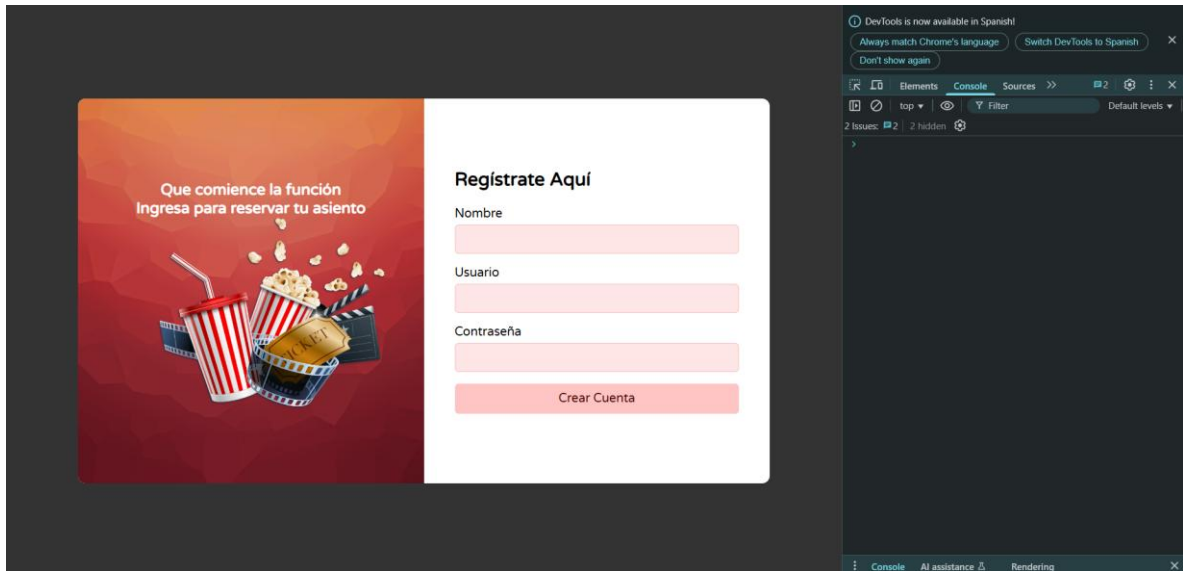
  try {
    const decoded = jwt.verify(token, process.env.SECRET_KEY);
    req.user = decoded; // guardar el usuario decodificado en el request
    console.log("Token decodificado:", decoded);
    next();
  } catch (error) {
    return res.status(403).json({ mensaje: "Token inválido o expirado" });
  }
};

module.exports = { verificarAdmin, verificarToken };
```

En este Middleware solo se verifica si el usuario que intenta realizar determinada acción cuenta tanto con permisos de Admin, si es que la acción que desea realizar lo requiere y siempre estará verificando que cada usuario que ingrese a la App cuente con su JWT. El JWT como tal se genera en el Login, aquí ya solo se verifica y valida.

Frontend





CineMax

Selecciona tu película y reserva tus asientos

- Todas
- Estrenos
- Acción
- Aventura
- Drama
- Ciencia Ficción

Avengers

Avengers: Endgame

2h 58m ★ 4.8/5

Acción Aventura
Ciencia Ficción

Seleccionar asientos

Dune

Dune: Parte Dos

2h 46m ★ 4.7/5

Ciencia Ficción Aventura
Drama

Seleccionar asientos

Oppenheimer

Oppenheimer

3h ★ 4.9/5

Drama Histórico
Biografía

Seleccionar asientos

Spider-Man

Spider-Man: No Way Home

2h 28m ★ 4.8/5

Acción Aventura
Superhéroes

Seleccionar asientos

The Batman

The Batman

2h 56m ★ 4.5/5

Acción Drama Noir

Seleccionar asientos

Jurassic World

Jurassic World: Dominion

2h 27m ★ 3.9/5

Acción Aventura
Ciencia Ficción

Seleccionar asientos